

Tabular data manipulation with Python I

ReproRehab Pod2

Tabular data

One common data format

	Variable X	Variable Y	Variable Z
Participant 1	...	...	...
Participant 2	...	...	...
Participant 3	...	...	...

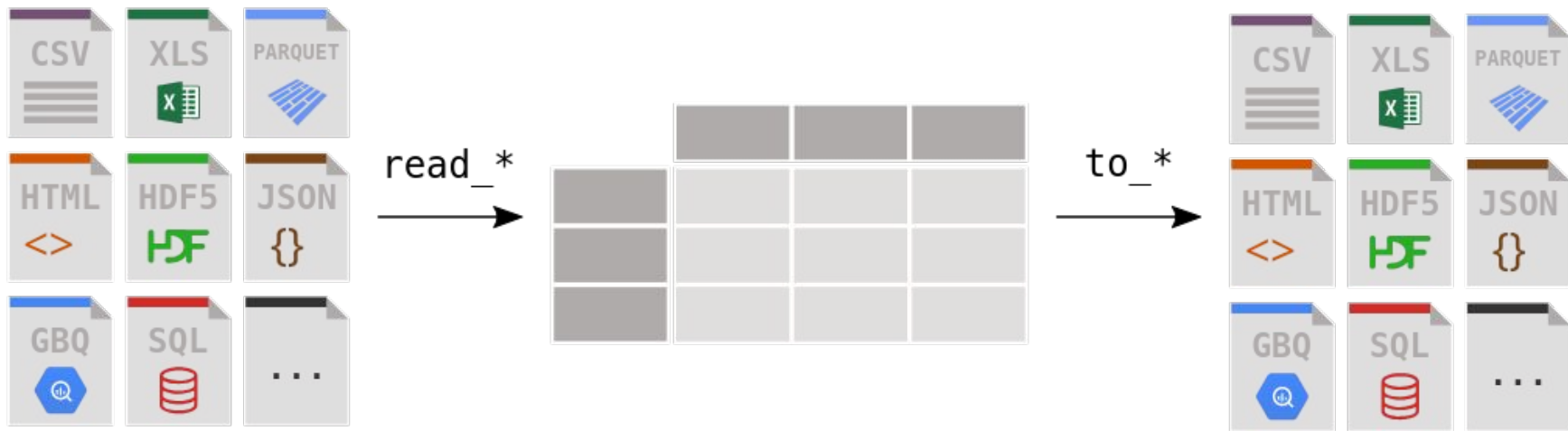
File extensions: .csv (comma separated values) / .tsv (tab separated values)

Pandas



pandas

Python package to handle tabular data.



Two basic data structures

DataFrame: a two-dimensional data structure that holds data like a two-dimension array or a table with rows and columns

Series: a one-dimensional labeled array holding data of any type

pandas Series and DataFrame					
datagy.io					
Series		Series		DataFrame	
Website		Visited		Website	Visited
0	datagy.io	0	2023-01-23	0	datagy.io 2023-01-23
1	google.com	1	2023-01-24	1	google.com 2023-01-24
2	bing.com	2	2023-01-04	2	bing.com 2023-01-04

Let's start with pandas

1. Type the command and run in the console:

```
In [1]: !pip install pandas
```

2. Load the package:

```
import pandas as pd
```

3. Load a sample data: 'mental_health_wearable_data.csv' using `pandas.read_csv()`

```
# %% Read sample data
data = pd.read_csv('mental_health_wearable_data.csv', sep=',')
data
```

Your first loaded tabular data

You can check that this is your loaded data.

You data in MS Excel

Heart_Rate_BPM	Sleep_Duration_Hours	Physical_Activity_Steps	Mood_Rating	Mental_Health_Condition
98	7.42512361767257	13760	5	1
111	9.45757234666567	11455	9	0
88	4.03710293584538	9174	8	1
74	8.86410142202555	12612	1	1
102	9.69021731673351	3923	9	0
67	4.54795440600134	12446	5	1

You data in Python/Spyder

	Heart_Rate_BPM	...	Mental_Health_Condition
0	98	...	1
1	111	...	0
2	88	...	1
3	74	...	1
4	102	...	0
...	...	...	...

You can open your DataFrame by clicking the variable

Name ▲	Type	Size	
data	DataFrame	[10000, 5]	Column names: Heart_Rate_BPM,

	Heart_Rate_BPM	Sleep_Duration_Hours	Physical_Activity_Steps	Mood_Rating	Mental_Health_Condition
0	98	7.42512	13760	5	1
1	111	9.45757	11455	9	0
2	88	4.0371	9174	8	1
3	74	8.8641	12612	1	1
4	102	9.69022	3923	9	0
5	67	4.54795	12446	5	1

Let's understand your loaded data

1. Index.
It starts from 0
2. Column names: if the same values are repeated, “.1”, “.2”, will be used to make each column name distinct (ex. Heart_Rate_BPM.1)

	Heart_Rate_BPM	...	Sleep_Duration_Hours	Physical_Activity_Steps	Mental_Health_Condition
0	98	...	1	0	1
1	111	...	0	1	0
2	88	...	1	1	1
3	74	...	1	1	1
4	102	...	0	0	0
...	...	...	...	...	...
9995	107	...	1	1	1
9996	82	...	0	0	0
9997	65	...	0	0	0
9998	110	...	1	1	1
9999	112	...	0	0	0

[10000 rows x 5 columns]

2-1. If you want to see the full list of columns, run {your DataFrame's name}.columns (ex. data.columns)

```
In [3]: data.columns
Out[3]:
Index(['Heart_Rate_BPM', 'Sleep_Duration_Hours', 'Physical_Activity_Steps',
      'Mood_Rating', 'Mental_Health_Condition'],
      dtype='object')
```

3. Size of your dataset

How to use .read_csv()

It's a versatile function with lots of options that can be configured. We will focus on a few.

filepath: **Path** to the file you read.
If the file you want to read is in your *working directory*, you can provide the name only. If not, you should provide the full path. (ex. `'/Users/joh/Documents/some_file.csv'`).

pandas.read_csv

```
pandas.read_csv(filepath_or_buffer *, sep=<no_default>, delimiter=None,
header='infer', names=<no_default>, index_col=None, usecols=None,
dtype=None, engine=None, converters=None, true_values=None,
false_values=None, skipinitialspace=False, skiprows=None, skipfooter=0,
nrows=None, na_values=None, keep_default_na=True, na_filter=True,
verbose=<no_default>, skip_blank_lines=True, parse_dates=None,
infer_datetime_format=<no_default>, keep_date_col=<no_default>,
date_parser=<no_default>, date_format=None, dayfirst=False,
cache_dates=True, iterator=False, chunksize=None, compression='infer',
thousands=None, decimal='.', lineterminator=None, quotechar='"', quoting=0,
doublequote=True, escapechar=None, comment=None, encoding=None,
encoding_errors='strict', dialect=None, on_bad_lines='error',
delim_whitespace=<no_default>, low_memory=True, memory_map=False,
float_precision=None, storage_options=None, dtype_backend=<no_default>)
```


How to use .read_csv()

It's a versatile function with lots of options that can be configured. We will focus on a few.

sep/delimiter: Character or regex that separates values. Possible values can be whitespace (' '), comma (','), tab('\t').

pandas.read_csv

```
pandas.read_csv(filepath_or_buffer *, sep=<no_default>, delimiter=None,
header='infer', names=<no_default>, index_col=None, usecols=None,
dtype=None, engine=None, converters=None, true_values=None,
false_values=None, skipinitialspace=False, skiprows=None, skipfooter=0,
nrows=None, na_values=None, keep_default_na=True, na_filter=True,
verbose=<no_default>, skip_blank_lines=True, parse_dates=None,
infer_datetime_format=<no_default>, keep_date_col=<no_default>,
date_parser=<no_default>, date_format=None, dayfirst=False,
cache_dates=True, iterator=False, chunksize=None, compression='infer',
thousands=None, decimal='.', lineterminator=None, quotechar='\"', quoting=0,
doublequote=True, escapechar=None, comment=None, encoding=None,
encoding_errors='strict', dialect=None, on_bad_lines='error',
delim_whitespace=<no_default>, low_memory=True, memory_map=False,
float_precision=None, storage_options=None, dtype_backend=<no_default>)
```

How to use .read_csv()

It's a versatile function with lots of options that can be configured. We will focus on a few.

header: Row number containing the column labels. If no value is provided for **names**, its value will be 0, or the first row.

names: Sequence of column labels to apply.

pandas.read_csv

```
pandas.read_csv(filepath_or_buffer *, sep=<no_default>, delimiter=None,
header='infer', names=<no_default>, index_col=None, usecols=None,
dtype=None, engine=None, converters=None, true_values=None,
false_values=None, skipinitialspace=False, skiprows=None, skipfooter=0,
nrows=None, na_values=None, keep_default_na=True, na_filter=True,
verbose=<no_default>, skip_blank_lines=True, parse_dates=None,
infer_datetime_format=<no_default>, keep_date_col=<no_default>,
date_parser=<no_default>, date_format=None, dayfirst=False,
cache_dates=True, iterator=False, chunksize=None, compression='infer',
thousands=None, decimal='.', lineterminator=None, quotechar='"', quoting=0,
doublequote=True, escapechar=None, comment=None, encoding=None,
encoding_errors='strict', dialect=None, on_bad_lines='error',
delim_whitespace=<no_default>, low_memory=True, memory_map=False,
float_precision=None, storage_options=None, dtype_backend=<no_default>)
```

How to use .read_csv()

It's a versatile function with lots of options that can be configured. We will focus on a few.

usecols: Subset of columns to select, denoted either by column indices or labels.

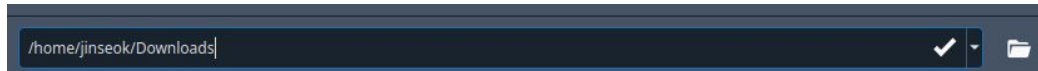
engine: Parser engine to use. If you want to read large files fast, use 'c' or 'pyarrow'.

pandas.read_csv

```
pandas.read_csv(filepath_or_buffer *, sep=<no_default>, delimiter=None,
header='infer', names=<no_default>, index_col=None, usecols=None,
dtype=None, engine=None, converters=None, true_values=None,
false_values=None, skipinitialspace=False, skiprows=None, skipfooter=0,
nrows=None, na_values=None, keep_default_na=True, na_filter=True,
verbose=<no_default>, skip_blank_lines=True, parse_dates=None,
infer_datetime_format=<no_default>, keep_date_col=<no_default>,
date_parser=<no_default>, date_format=None, dayfirst=False,
cache_dates=True, iterator=False, chunksize=None, compression='infer',
thousands=None, decimal='.', lineterminator=None, quotechar='\"', quoting=0,
doublequote=True, escapechar=None, comment=None, encoding=None,
encoding_errors='strict', dialect=None, on_bad_lines='error',
delim_whitespace=<no_default>, low_memory=True, memory_map=False,
float_precision=None, storage_options=None, dtype_backend=<no_default>)
```

All in context: filepath

Change your working directory here!



Our original command:

```
# %% Read sample data
data = pd.read_csv('mental_health_wearable_data.csv', sep=',')
```

Note. If you see this error, check if your working directory is the same as the folder that has 'mental_health_wearable_data.csv'.

```
FileNotFoundError: [Errno 2] No such file or directory: 'mental_health_wearable_data.csv'
```

Provide the full path to prevent this error:

```
data = pd.read_csv('/home/jinseok/Downloads/mental_health_wearable_data.csv',
                    sep=',')
```

Or provide the folder path first and then put together!

```
csv_folder = '/home/jinseok/Downloads/'
data = pd.read_csv(csv_folder + 'mental_health_wearable_data.csv', sep=',')
```

All in context: header/names

If you know the configuration of the file you read, that's good. If not, just check immediately and see if the column headers are correct.

```
In [3]: data = pd.read_csv('/home/jinseok/Downloads/mental_health_wearable_data.csv',  
...:                        sep=',')
```

```
In [4]: data
```

 Immediately checking the data after load

```
Out[4]:
```

	Heart_Rate_BPM	...	Mental_Health_Condition
0	98	...	1
1	111	...	0
2	88	...	1
3	74	...	1
4	102	...	0
...	...	...	...
9995	107	...	1
9996	82	...	0
9997	65	...	0
9998	110	...	1
9999	112	...	0

Do column names look correct?

```
[10000 rows x 5 columns]
```

```
In [82]: data.columns
```

```
Out[82]:
```

```
Index(['Heart_Rate_BPM', 'Sleep_Duration_Hours', 'Physical_Activity_Steps',  
      'Mood_Rating', 'Mental_Health_Condition'],  
      dtype='object')
```

data.columns can be useful!

All in context: header/names

If you know the names of the columns, and would like to load using different column names, you can use the following command:

```
data = pd.read_csv('mental_health_wearable_data.csv', header=0,  
                  names=['my_column_A', 'my_column_B',  
                        'my_column_C', 'my_column_D', 'my_column_E'])
```

If you skip `header=0`, this will happen:

```
In [9]: data  
Out[9]:
```

	my_column_A	...	my_column_E
0	Heart_Rate_BPM	...	Mental_Health_Condition
1	98	...	1
2	111	...	0
3	88	...	1
4	74	...	1
...	...	...	...
9996	107	...	1
9997	82	...	0
9998	65	...	0
9999	110	...	1
10000	112	...	0

Original column header is now read as the first data row, which is not what you want.

All in context: header/names

What if you want to name the columns after you load data?

Use `pandas.DataFrame.rename()`. You need to provide a mapping dictionary

ex. A data frame:

```
In [16]: data.head()
Out[16]:
```

	Heart_Rate_BPM	Sleep_Duration_Hours	...
0	98	7.425124	...
1	111	9.457572	...
2	88	4.037103	...
3	74	8.864101	...
4	102	9.690217	...

pre

```
In [18]: data.head()
Out[18]:
```

	HR_BPM	Sleep_Duration_Hours	...
0	98	7.425124	...
1	111	9.457572	...
2	88	4.037103	...
3	74	8.864101	...
4	102	9.690217	...

post

Mapping dict: {Current column name: New column name}

```
data.rename(columns={'Heart_Rate_BPM': 'HR_BPM'}, inplace=True)
```

Don't forget this!

All in context: header/names

You want to change all names?

```
In [20]: data.rename(columns={x:y for x, y in zip(data.columns, ['A', 'B', 'C', 'D', 'E'])}, inplace=True)
```

```
In [21]: data.head()
```

It's a dict comprehension, remember?

```
Out[21]:
```

	A	B	C	D	E
0	98	7.425124	13760	5	1
1	111	9.457572	11455	9	0
2	88	4.037103	9174	8	1
3	74	8.864101	12612	1	1
4	102	9.690217	3923	9	0

Mapping dict: {Current column name: New column name}

Side note: zip

`zip(iterable1, iterable2, ...)` combines elements of the iterables.

In python, '+' can be used to combine strings.
So the first `x + y` `>> 'Hello ' + 'World'`, and the
second `x + y` `>> 'I am ' + 'Jin'`.



```
[x + y for x, y in zip(['Hello', 'I am'], ['World', 'Jin'])]  
['Hello World', 'I am Jin']
```

More than TWO iterables can be zipped.
Here, x, y, z, p are the four elements from
four lists!

First x, y, z, p Next x, y, z, p



```
[' '.join((x, y, z, p)) for x, y, z, p in zip(['my', 'your'], ['name', 'names'], ['is', 'are'], ['Jin', 'Tom and Jerry'])]  
['my name is Jin', 'your names are Tom and Jerry']
```

.join is a method to combine strings using a specified separator. Here we're joining strings with whitespace in between. So for each combination of x, y, z, p, they will be combined with whitespace.

ex. `' '.join(('my', 'name', 'is', 'Jin'))` `>> 'my name is Jin'`

ex2. `'+'.join(('my', 'name', 'is', 'Jin'))` `>> 'my+name+is+Jin'`

Side note: zip

So in the previous example...

```
In [20]: data.rename(columns={x:y for x, y in zip(data.columns, ['A', 'B', 'C', 'D', 'E'])}, inplace=True)
```

```
In [21]: data.head()
```

```
Out[21]:
```

	A	B	C	D	E
0	98	7.425124	13760	5	1
1	111	9.457572	11455	9	0
2	88	4.037103	9174	8	1
3	74	8.864101	12612	1	1
4	102	9.690217	3923	9	0

It's a dict comprehension, remember?

```
In [82]: data.columns
```

```
Out[82]:
```

```
Index(['Heart_Rate_BPM', 'Sleep_Duration_Hours', 'Physical_Activity_Steps',  
      'Mood_Rating', 'Mental_Health_Condition'],  
      dtype='object')
```

(x, y) in each iteration will be:

The dictionary will be:

```
>> x: 'Heart_Rate_BPM', y: 'A'      (1st)
>> x: 'Sleep_Duration_Hours', y: 'B' (2nd)
>> x: 'Physical_Activity_Steps', y: 'C' (3rd)
>> x: 'Mood_Rating', y: 'D'         (4th)
>> x: 'Mental_Health_Condition', y: 'E' (5th)
```

```
{'Heart_Rate_BPM': 'A',
 'Sleep_Duration_Hours': 'B',
 'Physical_Activity_Steps': 'C',
 'Mood_Rating': 'D',
 'Mental_Health_Condition': 'E'}
```

'key' will be
renamed to
'value'

All in context: sep/delimiter

`sep` needs to be provided if you read files with extensions other than `.csv`. See this example:

sample_tsv.tsv

```
172.3    273.1    3829.1
378.1    311.2    7873.1
199.7    257.1    8812.6
```

No `sep` value provided

```
tsv_data = pd.read_csv(
    '/home/jinseok/Downloads/sample_tsv.tsv',
    header=None)
```

Single column data frame

```
In [49]: tsv_data
Out[49]:
```

0	172.3\t273.1\t3829.1
1	378.1\t311.2\t7873.1
2	199.7\t257.1\t8812.6

Only 1 column name

tsv_data	DataFrame	[3, 1]	Column names: 0
----------	-----------	--------	-----------------

This is original data: 3 columns

`sep='\t'` provided

```
tsv_data_with_sep = pd.read_csv(
    '/home/jinseok/Downloads/sample_tsv.tsv',
    sep='\t',
    header=None)
```

tsv_data_with_sep	DataFrame	[3, 3]	Column names: 0, 1, 2
-------------------	-----------	--------	-----------------------

```
In [50]: tsv_data_with_sep
Out[50]:
```

	0	1	2
0	172.3	273.1	3829.1
1	378.1	311.2	7873.1
2	199.7	257.1	8812.6

Three-column data frame

All in context: usecols

If you only need certain columns, use `usecols`.

```
In [10]: data = pd.read_csv('mental_health_wearable_data.csv',
...:                        usecols=['Heart_Rate_BPM', 'Mental_Health_Condition'])

In [11]: data
Out[11]:
```

	Heart_Rate_BPM	Mental_Health_Condition
0	98	1
1	111	0
2	88	1
3	74	1
4	102	0
...	...	...

Only the pre-defined columns are loaded

You can use a list of integers corresponding to the indices of the columns to be loaded.

```
data = pd.read_csv('mental_health_wearable_data.csv', usecols=[0,4])
```

All in context: engine & sep/delimiter.

If the file you're reading is several GB, `engine='pyarrow'` or `engine='c'` will speed up the loading process:

```
data = pd.read_csv('/home/jinseok/Downloads/mental_health_wearable_data.csv',  
                  engine='pyarrow')
```

You first need to install 'pyarrow'. `!pip install pyarrow`

Data frame: Basics

`pandas.DataFrame.shape`

-> returns a tuple: (# row, # column)

```
In [52]: data = pd.read_csv('mental_health_wearable_data.csv', sep=',')
...: data.shape
Out[52]: (10000, 5)
```

`len(pandas.DataFrame)`

-> returns integer: # row

```
In [53]: len(data)
Out[53]: 10000
```

`pandas.DataFrame[column label]`

-> returns pandas.Series

`pandas.DataFrame.column_label` also accepted
(No quotation marks!)

```
In [66]: data["Sleep_Duration_Hours"]
Out[66]:
0      7.425124
1      9.457572
2      4.037103
3      8.864101
4      9.690217
...
9995    7.517607
9996    8.195159
9997    7.151055
9998    9.008428
9999    4.198801
Name: Sleep_Duration_Hours, Length: 10000, dtype: float64
```

```
In [69]: data.Sleep_Duration_Hours
Out[69]:
0      7.425124
1      9.457572
2      4.037103
3      8.864101
4      9.690217
...
```

Data frame: Basics

pandas.DataFrame.dtypes

-> data type (ex. float64, str) of the column

```
In [83]: data.dtypes
Out[83]:
Heart_Rate_BPM          int64
Sleep_Duration_Hours    float64
Physical_Activity_Steps  int64
Mood_Rating             int64
Mental_Health_Condition  int64
dtype: object
```

pandas.DataFrame.describe()

-> generate descriptive statistics

```
In [76]: data.describe()
Out[76]:
```

	Heart_Rate_BPM	...
count	10000.000000	...
mean	89.822000	...
std	17.309144	...
min	60.000000	...
25%	75.000000	...
50%	90.000000	...
75%	105.000000	...
max	119.000000	...

pandas.Series.value_counts()

-> Counts of unique values

```
In [81]: data.Mood_Rating.value_counts()
Out[81]:
Mood_Rating
5      1178
9      1145
6      1139
7      1134
3      1123
8      1105
4      1091
1      1059
2      1026
Name: count, dtype: int64
```


Data frame: Basics

`pandas.DataFrame.head(n=5)`

-> prints the first n(default: 5) rows

`pandas.DataFrame.tail(n=5)`

-> prints the last n(default: 5) rows

```
In [29]: data.head()
Out[29]:
```

	Heart_Rate_BPM	Sleep_Duration_Hours	...
0	98	7.425124	...
1	111	9.457572	...
2	88	4.037103	...
3	74	8.864101	...
4	102	9.690217	...

`pandas.DataFrame.info()`

-> Prints a concise summary

```
In [30]: data.tail(10)
Out[30]:
```

	Heart_Rate_BPM	...
9990	104	...
9991	93	...
9992	80	...
9993	115	...
9994	114	...
9995	107	...
9996	82	...
9997	65	...
9998	110	...
9999	112	...

```
In [31]: data.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 5 columns):
 #   Column                                Non-Null Count  Dtype  
---  -
 0   Heart_Rate_BPM                        10000 non-null  int64  
 1   Sleep_Duration_Hours                  10000 non-null  float64
 2   Physical_Activity_Steps               10000 non-null  int64  
 3   Mood_Rating                           10000 non-null  int64  
 4   Mental_Health_Condition               10000 non-null  int64  
dtypes: float64(1), int64(4)
memory usage: 390.8 KB
```


Data frame: Basics

`pandas.DataFrame.mean()`

`pandas.DataFrame.max()`

`pandas.DataFrame.min()`

`pandas.DataFrame.median()`

`pandas.DataFrame.std()`

`pandas.DataFrame.abs()` -> convert to absolute values

`pandas.DataFrame.pivot_table()`

Data frame: Indexing (.iloc)

`pandas.DataFrame.iloc[row, column]` -> returns a slice of the data frame. Row / column is an integer, or a slice (noted by ':').

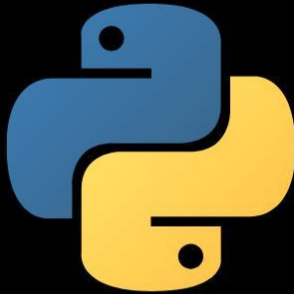
```
In [54]: data.iloc[0, 1]
Out[54]: np.float64(7.425123617672569)
```

```
In [57]: data.iloc[2, 2:]
Out[57]:
Physical_Activity_Steps  Mood_Rating  Mental_Health_Condition
0                    13760                5                    1
1                    11455                9                    0
```

Index	Heart_Rate_BPM	Sleep_Duration_Hours	Physical_Activity_Steps	Mood_Rating	Mental_Health_Condition
0	98	7.42512	13760	5	1
1	111	9.45757	11455	9	0
2	88	4.0371	9174	8	1
3	74	8.8641	12612	1	1
4	102	9.69022	3923	9	0

Index & Slice in Python

HOW SLICING IN PYTHON WORKS?



```
      +---+---+---+---+---+---+
      | P | y | t | h | o | n |
      +---+---+---+---+---+---+
Slice position: 0  1  2  3  4  5  6
Index position:  0  1  2  3  4  5

1  p = ['P','y','t','h','o','n']
2  print(p[0]) # Output: P
3  print(p[5]) # Output: n
4  print(p[0:1]) # Output: ['P']
5  print(p[0:1]) # Output: ['P']
6  print(p[0:2]) # Output: ['P','y']
```

www.codewithfaraz.com



Data frame: Indexing (.loc)

`pandas.DataFrame.loc[row, column]`

-> returns a slice of the data frame.

Use labels or boolean (True / False)

`':'` means 'all rows'

```
In [64]: data.loc[:, "Sleep_Duration_Hours"]  
Out[64]:  
0      7.425124  
1      9.457572  
2      4.037103  
3      8.864101  
4      9.690217  
...
```



Index	Heart_Rate_BPM	Sleep_Duration_Hours	Physical_Activity_Steps	Mood_Rating	Mental_Health_Condition
0	98	7.42512	13760	5	1
1	111	9.45757	11455	9	0
2	88	4.0371	9174	8	1
3	74	8.8641	12612	1	1
4	102	9.69022	3923	9	0

Data frame: Indexing (.loc)

pandas.DataFrame.loc[row, column]

```
In [65]: data.loc[data["Heart_Rate_BPM"] > 105, ["Sleep_Duration_Hours", "Mental_Health_Condition"]]  
Out[65]:  
Sleep_Duration_Hours  Mental_Health_Condition  
1          9.457572              0  
8          7.225241              0  
14         9.972529              0  
20         9.864739              0  
27         7.091002              1  
...
```

Index	Heart_Rate_BPM	Sleep_Duration_Hours	Physical_Activity_Steps	Mood_Rating	Mental_Health_Condition
0	98 > 105? False	7.42512	13760	5	1
1	111 > 105? True	9.45757	11455	9	0
2	88 > 105? False	4.0371	9174	8	1
3	74 > 105? False	8.8641	12612	1	1
4	102 > 105? False	9.69022	3923	9	0

Simple visualization with pandas

```
# %% DataFrame.plot

# Histogram
data.plot.hist(column='Heart_Rate_BPM',
               alpha=0.6, edgecolor='k',
               facecolor='pink', title='Heart Rate BPM', legend=False)

# Boxplot
data.plot.box(column='Heart_Rate_BPM', by="Mood_Rating",
              title='Heart Rate BPM by Mood Rating',
              xlabel='Mood Rating')

# Scatterplot
data.plot.scatter(x='Heart_Rate_BPM', y='Sleep_Duration_Hours',
                 s=4, edgecolor='k', color='None')

# Hexbin
data.plot.hexbin(x="Heart_Rate_BPM", y='Sleep_Duration_Hours',
                 gridsize=30, cmap='cividis')
```